

Choix d'un ESP transactionnel pour Attendee LFD

Ce rapport met à jour et complète votre note de travail existante, en la recentrant sur une décision opérationnelle entre Mailgun, Brevo et Postmark pour un cas d'usage transactionnel à enjeu public français. `filecite0turn0file0`

Synthèse exécutive

Pour **Attendee/LFD**, avec **20 000 emails sur deux jours**, des **pics correspondant à environ 3 000 inscriptions quasi simultanées**, un besoin de **mise en route rapide**, une stack **NestJS/Node.js**, et une **sensibilité forte à la souveraineté**, le meilleur compromis global parmi ces trois acteurs est **Mailgun en région EU, sur IP partagée, via API HTTP et file d'attente applicative**. La raison n'est pas seulement technique : Mailgun documente explicitement un **choix de région UE ou US**, avec des **message data, logs, suppressions, statistiques et IPs "region-bound"** qui "ne quittent jamais" la région de traitement choisie, tout en restant compatible avec un compte unique. En parallèle, Mailgun recommande officiellement les **IP partagées** pour les volumes **inférieurs à 100 000 emails/mois** ou volatils, ce qui correspond exactement à votre profil, et son propre site indique que **l'API HTTP est plus fiable à l'échelle** que le SMTP. Côté budget, le scénario 20k reste bas : **\$33/mois** sur le plan Basic ou **\$35/mois** sur Foundation si vous voulez davantage de rétention de logs et garder l'option d'un plan plus "procurement-friendly". ¹

Brevo arrive comme **alternative politique et juridique crédible** si la priorité du ministère est d'abord le fait d'avoir un **éditeur français**, avec entité contractante parisienne, DPA intégré aux conditions, clause de notification si le service ou la résidence des données sortent de l'EEE, et un SLA public à **99,8 % de disponibilité mensuelle moyenne**. Techniquement, Brevo est aussi solide pour vos pics : sa documentation publique affiche **1 000 requêtes/seconde** sur `POST /v3/smtp/email` dans le palier général, avec recommandation de lisser les appels. En revanche, son positionnement public n'est pas aussi propre que Mailgun sur la **résidence des données de message** : la politique de confidentialité indique des **opérations aux États-Unis et dans l'Union européenne**, avec possibilité de transferts transfrontaliers, et la documentation publique revue ici ne montre **pas de sélecteur de région client aussi explicite** que Mailgun. Il faut aussi intégrer une **friction d'onboarding** : Brevo écrit qu'il faut d'abord que le compte soit **approuvé pour l'envoi** avant de pouvoir envoyer au-delà de la simple création de compte. ²

Postmark est probablement le plus agréable pour une équipe Node/NestJS purement transactionnelle : sa documentation TypeScript/Node est très lisible, ses bibliothèques officielles sont présentées comme le moyen "le plus simple" d'intégration, il expose des webhooks propres, un endpoint batch jusqu'à **500 messages par appel**, et il pousse fortement une culture de délivrabilité transactionnelle. Mais pour un **cadre "niveau ministère"**, le point bloquant est frontal : Postmark indique que ses **serveurs primaires sont hébergés hors de Chicago** chez Deft et sur AWS, et qu'il **n'a pas de projet d'ajouter des serveurs dans l'UE**. Autrement dit, même si Postmark est RGPD-compatible via DPA/SCC, il est **nettement moins défendable** si votre lecture de la souveraineté exige une **résidence UE** au minimum. ³

En une phrase : **Mailgun** est le meilleur choix si vous devez défendre un compromis sérieux entre **résidence UE, robustesse transactionnelle, pics d'envoi, coût faible et effort d'intégration raisonnable** ; **Brevo** devient le meilleur choix si le critère dominant est **"prestataire français"** et si vous

acceptez une position publique moins nette sur la segmentation de résidence des données de message ; **Postmark** ne doit être retenu que si la **souveraineté stricte n'est finalement pas un critère bloquant**. ⁴

Comparatif synthétique

Fournisseur	Fiabilité et délivrabilité	Gestion des pics et limites	IP partagée ou dédiée	Résidence des données et conformité	DX Node.js et NestJS
Mailgun	Positionné comme plateforme d'email delivery/deliverability à grande échelle, avec recommandation officielle de l' API HTTP plutôt que SMTP à l'échelle.	Pas de plafond RPS self-service public trouvé dans les sources revues, mais offre entreprise avec Rapid Fire Burst Sending SLA . Pour <100k/ mois ou volume irrégulier, Mailgun recommande les IP partagées .	IP partagée recommandée pour votre cas. IP dédiées accessibles à partir de 50k/ mois et incluses à certains niveaux supérieurs; warm-up automatisé documenté sur IP dédiée.	Choix explicite de région UE ou US . Les messages, logs, suppressions, stats, routes et IP restent dans la région choisie; certaines données de compte sont répliquées globalement. DPA Sinch/ GDPR disponible.	SDK Node officiel clair; si vous utilisez l'infra UE, il faut passer <code>url: 'https://api.eu.mailgun.net'</code> . Intégration NestJS simple via service injectable.

Fournisseur	Fiabilité et délivrabilité	Gestion des pics et limites	IP partagée ou dédiée	Résidence des données et conformité	DX Node.js et NestJS
Brevo	Suite plus large que purement transactionnelle, mais la page produit transactionnelle met en avant envoi en temps réel par API/SMTP et la gestion de logs; SLA public à 99,8% de disponibilité moyenne mensuelle.	Documentation publique rare mais utile : 1 000 RPS sur <code>POST /v3/smtp/email</code> au palier général, avec recommandation de répartir les appels uniformément; convient donc très bien à 3 000 inscriptions si vous queuez côté application.	La dedicated IP n'apparaît publiquement qu'en Enterprise ; les conditions indiquent une souscription annuelle séparée et que Brevo décide de l'attribuer ou non. Pour votre profil, l'usage implicite est l'infrastructure partagée.	Entité contractante française, DPA intégré, notification prévue si service/ résidence sort de l'EEE. Mais la privacy policy mentionne des opérations en UE et aux États-Unis et des transferts transfrontaliers possibles. Pas de sélecteur public de région comparable à Mailgun dans les sources revues.	Très correct en Node : SDK <code>@getbrevo/brevo</code> , compatibilité Node 18+, exemples API et SMTP. Bonne intégration NestJS, mais plateforme plus large donc plus verbeuse et moins "mono-usage transactionnel" que Postmark.
Postmark	Très fort ADN transactionnel, docs orientées dev, suivi des vitesses de livraison publié, et positionnement livré pour "transactional email".	Pas de plafond RPS public global trouvé, mais emails illimités/jour sur les plans payants et endpoint batch jusqu'à 500 messages par appel . Suffisant pour vos pics via queue + batch.	Postmark pousse fortement l' IP partagée , jugeant qu'elle donne souvent de meilleurs résultats. IP dédiée seulement à partir de 300 000/mois, \$50/IP/mois , warm-up géré ou DIY, réputation complète attendue en 3 à 6 semaines .	DPA/SCC et page RGPD disponibles, mais serveurs principaux hors Chicago et AWS , et aucun projet d'ajouter des serveurs UE . C'est le point faible majeur pour la souveraineté.	Excellente expérience Node/TypeScript; bibliothèques officielles présentées comme le moyen le plus simple d'intégration; très bon candidat pour NestJS si la souveraineté n'est pas bloquante.

Coûts et démarches d'onboarding

Pour votre volume, il faut raisonner en **coût réel du pic**, pas en "capacité entreprise théorique". Ici, 20 000 emails sur deux jours est un **petit volume mensuel** pour les trois fournisseurs. Le différenciateur économique n'est donc pas la facture brute, mais surtout **le niveau de résidence des données, la friction de démarrage, et le besoin — ou non — d'une IP dédiée.** ⁸

Fournisseur	Plan crédible pour 20k	Estimation de coût pour 20k	Option IP dédiée	Support public pertinent	Lecture achat public
Mailgun	Basic si l'objectif est le coût minimal; Foundation si vous voulez une position contractuelle plus "propre" et davantage de logs.	\$33/mois sur Basic = \$15 pour 10k inclus + \$18 d'overage pour 10k supplémentaires. \$35/mois sur Foundation pour 50k inclus.	\$59/IP/mois additionnelle une fois l'option dédiée accessible; inutile ici.	Ticket sur Basic; plus de support sur niveaux supérieurs.	Très bon ratio risque/prix. Foundation à \$35 est souvent le meilleur point d'équilibre procurement.

Fournisseur	Plan crédible pour 20k	Estimation de coût pour 20k	Option IP dédiée	Support public pertinent	Lecture achat public
Brevo	Starter ou PAYG , selon si vous voulez de l'abonnement ou des crédits non expirants.	Le montant exact 20k n'est pas exposé dans le crawl officiel de la grille tarifaire, bien que la page confirme un point d'entrée Starter à 5 000 emails/mois et l'existence d'un PAYG. Votre note interne précédemment positionnait Brevo dans un ordre de grandeur de quelques dizaines d'euros par mois sur ce type de volume, mais il faut reconfirmer le prix exact dans le configurateur/app avant validation d'achat.	IP dédiée seulement côté Enterprise et souscription annuelle séparée.	Support email sur Starter; pas de support téléphonique en PAYG; téléphone à partir de Professional.	Potentiellement économique, mais il faut exiger un chiffrage écrit avant arbitrage final.
Postmark	Pro est généralement le meilleur point d'entrée à 20k; Basic fonctionne aussi.	\$29.50/mois sur Pro = \$16.50 + \$13 pour 10k supplémentaires. En Alternative : \$33/mois sur Basic, ou \$30/mois sur Platform.	\$50/IP/mois , mais seulement si vous envoyez 300k/mois ou plus; donc hors sujet ici.	Support humain et docs très riches; pas de palier premium chiffré public trouvé.	Prix très compétitif, mais bloqué politiquement si la résidence UE est impérative.

Les calculs Mailgun et Postmark sont directement auditable à partir des grilles publiques. Pour Brevo, la structure est documentée publiquement — Starter dès 5 000 emails/mois, emails transactionnels sur tous les plans, crédits PAYG qui n'expirent pas — mais **la valeur monétaire exacte du cran 20k n'est pas visible textuellement dans la capture web officielle disponible ici**. Je déconseille donc d'inscrire un total "dur" pour Brevo dans une note d'arbitrage sans capture du configurateur ou devis commercial.

La table ci-dessous répond précisément à votre demande sur le **processus de région** et les **éventuels délais/approvals**.

Fournisseur	Choix de région	Retards, approbations ou dépendances documentées	Conséquence pratique
Mailgun	Oui , lors de l'ajout d'un domaine : il faut sélectionner la Domain region . Les données de message sont ensuite traitées en UE ou US selon ce choix.	Vérification du domaine via email + DNS; propagation DNS annoncée à 24-48 h . Le quickstart mentionne aussi un guided setup pour activer le compte.	Démarrage rapide si vous contrôlez votre DNS; il faut juste anticiper la vérification.
Brevo	Pas de sélecteur public de région de message data trouvé dans les sources revues; la localisation du service et de la donnée renvoie au DPA/Terms.	Brevo indique que l'on peut créer le compte immédiatement, mais qu'il faut ensuite que le compte soit approuvé pour l'envoi avant de réellement envoyer.	C'est la plus forte friction de démarrage côté shared flow.
Postmark	Non pour l'UE : la documentation RGPD explique qu'il n'y a pas de projet d'ajouter des serveurs dans l'UE .	Le démarrage de base est rapide et sans carte bancaire; en revanche, la dedicated IP suppose une demande et n'est de toute façon accessible qu'à partir de 300k/mois.	Très simple à démarrer techniquement, mais non aligné si la résidence UE est non-négociable.

Les preuves officielles sont assez claires : chez Mailgun, le choix de région est un **paramètre explicite de domaine** et la contrainte principale est le DNS; chez Brevo, la contrainte principale est une **approbation d'envoi**; chez Postmark, la contrainte principale n'est pas un délai mais l'**absence d'option UE** elle-même. ¹⁰

Architecture et charge

Sur la charge pure, vos chiffres ne justifient **ni IP dédiée**, ni warm-up d'IP privée accéléré, ni architecture exotique. Ce que vous avez, ce n'est pas un problème de volume mensuel massif; c'est un problème de **burst applicatif**. La bonne réponse n'est donc pas "acheter une IP dédiée", mais **désynchroniser l'envoi** de l'action d'inscription, **mettre les demandes dans une queue**, et laisser un worker NestJS pousser vers l'API transactionnelle du fournisseur avec réessais, idempotence et gestion des retours webhook. Cette recommandation est très cohérente avec les docs revues : Mailgun recommande l'API à l'échelle, Brevo expose des rate limits explicites sur l'endpoint transactionnel, et Postmark propose un endpoint batch pour les cas de volume ou de contrainte de temps de traitement.

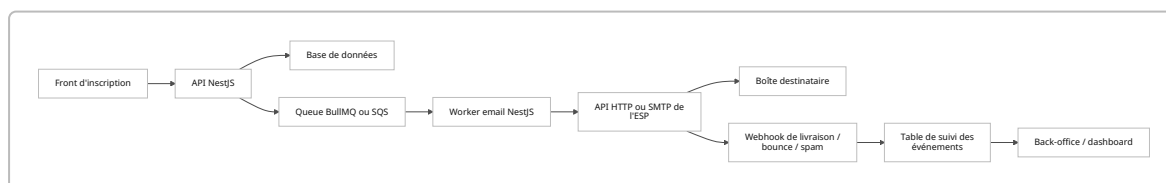
¹¹

Concrètement, **Brevo** est le seul des trois à publier ici une valeur de référence simple : **1 000 RPS** sur `POST /v3/smtp/email` au palier général, avec un exemple de **60 000 requêtes/minute si réparties uniformément**. Pour un pic de 3 000 inscriptions, cela veut dire qu'en théorie le fournisseur n'est pas le goulot si votre application ne fait pas n'importe quoi. **Postmark** ne publie pas un plafond global RPS

dans les sources revues, mais son `/email/batch` accepte **500 messages par appel**, ce qui signifie qu'un burst de 3 000 messages peut être compressé en **6 appels batch** au lieu de 3 000 appels unitaires. **Mailgun** ne publie pas ici de plafond RPS self-service, mais ses docs marketing/produit disent clairement que l'API est **plus fiable à l'échelle** que SMTP et son offre entreprise ajoute un cadre "Rapid Fire" pour les très gros bursts. ¹²

Sur la question **shared vs dedicated IP**, le bon choix pour vous est **shared** chez les trois. Chez Mailgun, la recommandation officielle est explicite : **partagée si vous envoyez moins de 100 000 messages/mois ou si votre volume est volatile**. Chez Postmark, la doctrine est encore plus nette : la **shared pool** est présentée comme meilleure pour la plupart des expéditeurs, et la dedicated IP n'est proposée qu'à partir de **300 000/mois**. Chez Brevo, la dedicated IP apparaît dans l'offre **Enterprise** et les conditions précisent qu'elle est séparément facturée sur base annuelle et accordée à la discrétion de l'éditeur ; pour un événement ponctuel à 20k/2 jours, ce n'est simplement pas la bonne couche économique ni opérationnelle. ¹³

Pour le **warm-up**, il faut bien distinguer **warm-up d'IP dédiée** et **échauffement prudent du domaine/sous-domaine d'envoi**. Les docs Mailgun et Postmark documentent très clairement que le warm-up d'une IP dédiée prend du temps et n'est pas conseillé comme solution miracle pour un envoi ponctuel ; Postmark indique même qu'un warm-up géré donne en général une réputation "full-volume" en **3 à 6 semaines**. Pour votre événement, la bonne approche est plutôt un **mini warm-up de domaine** : démarrer quelques jours avant avec des messages réellement transactionnels à audience très engagée, monter progressivement, puis réserver le pic à l'IP partagée du fournisseur. C'est une recommandation d'architecture raisonnable déduite de leurs guides, pas une exigence spécifique publiée mot pour mot pour votre cas. ¹⁴



Cette architecture découple la latence du parcours d'inscription de la latence fournisseur, absorbe les bursts, et permet d'exploiter les **webhooks** natifs des trois acteurs pour mettre à jour vos statuts métier sans faire de polling inutile. C'est particulièrement aligné avec la documentation Brevo, qui recommande les webhooks pour éviter de sursolliciter l'API, ainsi qu'avec les docs Mailgun et Postmark sur les événements de livraison. ¹⁵

Voici maintenant des **snippets NestJS/Node.js** minimaux. Ils sont volontairement simples : l'objectif est de vous montrer la friction d'intégration réelle.

Mailgun — API HTTP dans un service NestJS

```
import { Injectable } from '@nestjs/common';
import Mailgun from 'mailgun.js';
import FormData from 'form-data';

@Injectable()
export class MailgunEmailService {
  private readonly client = new Mailgun(FormData).client({
    username: 'api',
```

```

    key: process.env.MAILGUN_API_KEY!,
    url: 'https://api.eu.mailgun.net', // région EU
  });

  async sendConfirmation(to: string) {
    return this.client.messages.create(process.env.MAILGUN_DOMAIN!, {
      from: `Attendee <no-reply@${process.env.MAILGUN_DOMAIN}>`,
      to: [to],
      subject: 'Confirmation d'inscription',
      text: 'Votre inscription est confirmée.',
      html: '<p>Votre inscription est confirmée.</p>',
    });
  }
}

```

16

Mailgun — transport Nodemailer via SMTP relay

```

import * as nodemailer from 'nodemailer';

export const mailgunTransport = nodemailer.createTransport({
  host: process.env.MG_SMTP_HOST!, // ex. hôte SMTP de votre région
  depuis le dashboard
  port: Number(process.env.MG_SMTP_PORT ?? 587),
  secure: false,
  auth: {
    user: process.env.MG_SMTP_USER!, // souvent postmaster@votre-domaine
    pass: process.env.MG_SMTP_PASSWORD!,
  },
});

```

Le choix API vs SMTP n'est pas neutre : Mailgun écrit que **SMTP est simple**, mais que **l'API est plus fiable à l'échelle**, ce qui pousse plutôt à privilégier le premier snippet pour votre cas. ¹⁷

Brevo — API HTTP dans un service NestJS

```

import { Injectable } from '@nestjs/common';
import { BrevoClient } from '@getbrevo/brevo';

@Injectable()
export class BrevoEmailService {
  private readonly brevo = new BrevoClient({
    apiKey: process.env.BREVO_API_KEY!,
  });

  async sendConfirmation(to: string) {
    return this.brevo.transactionalEmails.sendTransacEmail({
      subject: 'Confirmation d'inscription',
    });
  }
}

```

```

    htmlContent: '<p>Votre inscription est confirmée.</p>',
    sender: { name: 'Attendee', email: 'no-reply@exemple.fr' },
    to: [{ email: to }],
  });
}
}

```

18

Brevo — transport Nodemailer via SMTP relay

```

import * as nodemailer from 'nodemailer';

export const brevoTransport = nodemailer.createTransport({
  host: process.env.BREVO_SMTP_HOST!, // hôte SMTP relay fourni dans vos
  // paramètres Brevo
  port: 587,
  secure: false,
  auth: {
    user: process.env.BREVO_SMTP_USER!,
    pass: process.env.BREVO_SMTP_KEY!, // clé SMTP, pas clé API
  },
});

```

Brevo documente que le **SMTP relay** utilise une **clé SMTP** dédiée, supporte **587/2525** en non chiffré et **465** en SSL/TLS, et ne supporte **pas** le batch; pour les batchs, il faut utiliser l'API. ¹⁹

Postmark — API dans un service NestJS

```

import { Injectable } from '@nestjs/common';
import { ServerClient } from 'postmark';

@Injectable()
export class PostmarkEmailService {
  private readonly client = new ServerClient(
    process.env.POSTMARK_SERVER_TOKEN!,
  );

  async sendConfirmation(to: string) {
    return this.client.sendEmail({
      From: 'no-reply@exemple.fr',
      To: to,
      Subject: 'Confirmation d'inscription',
      TextBody: 'Votre inscription est confirmée.',
      HtmlBody: '<p>Votre inscription est confirmée.</p>',
      MessageStream: 'outbound',
    });
  }
}

```

```
}  
}
```

20

Postmark — transport Nodemailer via SMTP

```
import * as nodemailer from 'nodemailer';  
  
export const postmarkTransport = nodemailer.createTransport({  
  host: process.env.POSTMARK_SMTP_HOST!,  
  port: Number(process.env.POSTMARK_SMTP_PORT ?? 587),  
  secure: false,  
  auth: {  
    user: process.env.POSTMARK_SMTP_USER!,  
    pass: process.env.POSTMARK_SMTP_PASSWORD!,  
  },  
});
```

Postmark met en avant une **intégration facile via API ou SMTP** et une bibliothèque Node qui s’installe en une commande. Pour une équipe NestJS, Postmark reste le plus “clean” côté developper experience pure, mais cela ne compense pas son déficit de souveraineté dans votre contexte. ²¹

Souveraineté et argumentaire ministère

Si vous devez défendre ce choix devant un ministère, il faut être extrêmement précis sur ce que vous entendez par **souveraineté**. Si le besoin veut dire “**éditeur français**”, alors **Brevo** est naturellement le plus présentable : société française, siège parisien, DPA et SLA intégrés dans les conditions, et documentation juridique orientée conformité européenne. Mais si le besoin veut dire “**résidence explicite des données de message dans l’UE, avec comportement public documenté**”, alors **Mailgun** est plus solide, parce qu’il dit noir sur blanc que les **message data** restent dans la région choisie et n’en sortent pas. Postmark, lui, est immédiatement affaibli sur ce terrain, car il indique publiquement ne pas prévoir de serveurs UE. ²²

Il faut aussi regarder la **nuance contractuelle**. Chez Mailgun/Sinch, le DPA précise qu’en fournissant des services de communication — y compris email — Sinch agit, pour certaines fonctions nécessaires de réseau, sécurité, anti-spam et maintenance, en **independent data controller** et non pas exclusivement en simple sous-traitant. Pour un acheteur public exigeant, ce n’est pas un “red flag” automatique, mais c’est un point qu’il faut **comprendre et assumer** dans la note de choix. À l’inverse, la documentation Brevo est plus rassurante politiquement pour un lecteur français, mais elle admet aussi des **opérations en UE et aux États-Unis** et des transferts transfrontaliers possibles. Sur Postmark, le cadre RGPD existe bien via DPA/SCC, mais il repose explicitement sur une infrastructure **non européenne**. ²³

Le risque opérationnel le plus sous-estimé n’est pas le coût. C’est la capacité à dire, dans une réunion achat/risk/compliance, **où vont les données de message et avec quel niveau de contrôle**. Sur ce point, la hiérarchie me paraît être la suivante. **Mailgun** est le meilleur sur la **clarté publique du modèle de résidence**. **Brevo** est le meilleur sur la **narration institutionnelle française**, mais moins clair publiquement sur la granularité de résidence de la donnée de message. **Postmark** est le meilleur sur la

qualité produit transactionnelle côté développeur, mais le moins défendable si le ministère exige une lecture stricte de la souveraineté numérique. ²⁴

Si vous avez besoin d'un **texte de justification** à réutiliser devant un interlocuteur ministère, celui-ci est le plus défendable juridiquement et techniquement :

Nous recommandons Mailgun en région EU, sur IP partagée, pour l'envoi des emails transactionnels d'Attendee/LFD. Ce choix répond à la contrainte de montée en charge sans surcoût inutile, tout en offrant une documentation publique explicite sur le traitement des données de message en région européenne. Le volume prévu, bien qu'avec des pics courts, reste trop faible et trop volatil pour justifier une IP dédiée. L'architecture recommandée s'appuie sur l'API HTTP du fournisseur, une file d'attente applicative et des webhooks de suivi, afin de garantir la robustesse au pic et la traçabilité des remises. Brevo demeure une alternative recevable si la priorité politique est de contracter avec un acteur français, mais sa documentation publique est moins explicite sur la segmentation régionale des messages data. Postmark, malgré une excellente qualité technique, n'est pas retenu à ce stade en raison de son absence d'option de résidence UE documentée. ²⁵

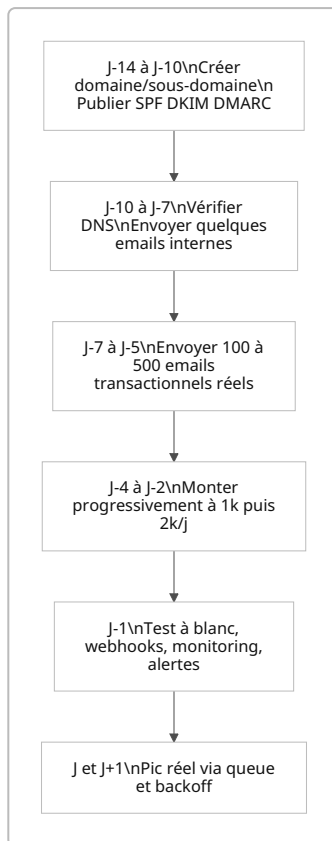
Recommandation finale et checklist

Ma recommandation finale est donc la suivante : **choisir Mailgun, région EU, IP partagée, API HTTP**, avec une **queue NestJS** et un **handler de webhooks**. C'est la seule option qui coche simultanément, de manière publiquement documentée, les cases **résidence UE explicite, coût très bas, compatibilité burst/prix**, et **position défendable devant une instance publique**. Si, pour des raisons institutionnelles, le sponsor impose un **éditeur français** comme critère premier, basculer vers **Brevo** est défendable à condition d'obtenir **par écrit** la réponse sur la localisation effective des messages data, la validation de l'onboarding et le support pendant la fenêtre de pic. **Postmark** doit rester le plan B technique si la souveraineté est requalifiée comme simple conformité RGPD et non comme résidence UE. ²⁶

La checklist opérationnelle la plus utile, dans l'ordre, tient en peu d'actions mais il faut la suivre sans improviser. D'abord, **créez un sous-domaine d'envoi dédié** du type `notify.votredomaine.fr` ou `mail.votredomaine.fr`, afin de séparer la réputation transactionnelle du domaine principal. Ensuite, **publiez et vérifiez les enregistrements DNS** demandés par le fournisseur : au minimum **SPF**, **DKIM**, et selon le fournisseur **CNAME** de tracking, **MX/Return-Path** si nécessaire; Mailgun documente précisément cette étape et rappelle que la propagation DNS peut prendre **24 à 48 h**. En parallèle, activez une politique **DMARC** adaptée, même minimale au départ, car Mailgun et Postmark documentent explicitement l'usage des standards d'authentification modernes. ²⁷

Ensuite, **ne faites jamais partir l'email depuis la requête HTTP d'inscription**. Enregistrez l'inscription, poussez un job dans **BullMQ/SQS**, envoyez depuis un worker, et traitez les événements `delivered`, `soft_bounce`, `hard_bounce`, `spam`, `blocked`, etc. par **webhook** dans votre base de suivi. Brevo documente explicitement ces événements transactionnels et recommande les webhooks pour éviter le polling; Mailgun et Postmark ont le même modèle orienté événements. ²⁸

Enfin, même si vous restez en IP partagée, je recommande un **échauffement court du domaine** selon une chronologie pragmatique. Une version simple est la suivante :



Cette timeline n'est pas une reprise littérale d'un guide fournisseur pour votre cas exact; c'est une traduction opérationnelle prudente à partir de leurs recommandations générales sur réputation, shared IP et warm-up des IP dédiées. ²⁹

Pour finir, voici un **message-type à envoyer au fournisseur choisi** avant engagement, afin d'obtenir une réponse exploitable en comité d'arbitrage :

Bonjour,

Nous préparons un dispositif transactionnel pour un service à contexte public français, avec environ 20 000 emails sur 48 h et des pics pouvant correspondre à 3 000 inscriptions quasi simultanées.

Nous avons besoin d'une confirmation écrite sur les points suivants :

1. région exacte de traitement et de stockage des **message data**, logs d'événements et suppressions ;
2. étapes d'**onboarding/approval** éventuelles avant mise en production ;
3. niveau de **throughput** recommandé pour notre fenêtre de pic et bonnes pratiques de burst ;
4. recommandation explicite entre **IP partagée** et **IP dédiée** pour ce profil ;
5. DPA, sous-traitants, durée de rétention des logs et procédure de suppression ;
6. support disponible pendant la fenêtre d'événement et canal d'escalade.

Merci de nous indiquer également s'il existe des contraintes spécifiques pour un organisme public français.

Si vous voulez une décision nette, courte et soutenable : **Mailgun EU + shared IP + API HTTP + queue applicative**. C'est la recommandation la plus robuste pour votre besoin réel. ³⁰

- 1 4 5 24 25 26 30 Regions - Transactional Email API Service for Developers - Transactional Email API Service For Developers | Mailgun
<https://www.mailgun.com/about/regions/>
- 2 22 Privacy Policy | Brevo
<https://www.brevo.com/legal/privacypolicy/>
- 3 Official Postmark libraries | Postmark Developer Documentation
<https://postmarkapp.com/developer/integration/official-libraries>
- 6 Brevo Terms of Service
<https://www.brevo.com/legal/termsfuse/>
- 7 Webhooks overview | Postmark Developer Documentation
<https://postmarkapp.com/developer/webhooks/webhooks-overview>
- 8 9 11 17 Flexible Email Delivery & API Pricing Plans - Transactional Email API Service For Developers | Mailgun
<https://www.mailgun.com/pricing/>
- 10 Getting started Sending Email
<https://documentation.mailgun.com/docs/mailgun/user-manual/domains/domains-custom>
- 12 15 Rate limits | Brevo API Documentation
<https://developers.brevo.com/docs/api-limits>
- 13 29 When do I need a dedicated IP address? – Mailgun Help Center
<https://help.mailgun.com/hc/en-us/articles/202453900-When-do-I-need-a-dedicated-IP-address>
- 14 Automatic IP Warmup – Mailgun Help Center
<https://help.mailgun.com/hc/en-us/articles/8305731694747-Automatic-IP-Warmup>
- 16 Node.js
https://documentation.mailgun.com/docs/mailgun/sdk/nodejs_sdk/
- 18 Node.js SDK | Brevo API Documentation
<https://developers.brevo.com/guides/node-js>
- 19 SMTP relay integration | Brevo API Documentation
<https://developers.brevo.com/docs/smtp-integration>
- 20 Sending email with API | Postmark Developer Documentation
<https://postmarkapp.com/developer/user-guide/send-email-with-api>
- 21 Send emails with Node.js
<https://postmarkapp.com/send-email/node>
- 23 Explore Data Processing Agreement - Sinch
<https://sinch.com/legal/terms-and-conditions/other-sinch-terms-conditions/data-protection-agreement/>
- 27 Getting started Sending Email
<https://documentation.mailgun.com/docs/mailgun/user-manual/domains/domains-verify>
- 28 Transactional webhooks | Brevo API Documentation
<https://developers.brevo.com/docs/transactional-webhooks>